

# aTOMos v3.6.1.6

## Literal self-referential kernel

ASA/NA + J–K state feedback  
with editable equation inputs

Explicit recurrence · Exact Boolean algebra  
CPU / CUDA execution · Independent replay

### Tom Klootwijk

|                            |                                |
|----------------------------|--------------------------------|
| Author-supplied identifier | NL200678942                    |
| Author-supplied date       | 10-07-1990                     |
| Base engine                | aTOMos 3.6.1 / M2              |
| Imported prototype         | UGTS-KC 3.6.2 / SCLP           |
| Target                     | RTX 5070 Ti Laptop GPU / 12 GB |
| CUDA target                | sm_120 + compute_120           |
| Release                    | 3.6.1.6 / 14 September 2026    |

#### An executable state-feedback subversion

Literal ASA/NA and J–K equations now form a persistent recurrence. Editable expressions compile to exact truth tables; every intermediate state is replayed independently. The SATNAV and UGTS numerical profiles remain included.

The package contains the formal specification, CUDA and CPU source, an independent reference, test inputs and actual verification records. The source's geometric definitions and the new satnav model are distinguished throughout.

## Release and implementation record

This subversion completes the literal ASA/NA and J–K primitives with explicit state feedback, supplied Boolean equations and independent replay. The new **SRK-R1** profile is formally specified in Section 1. It also retains the 3.6.1.5 SATNAV-R1 corrected-code and UGTS implementations.

| Release item           | Disposition                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Subversion             | <b>3.6.1.6</b> ; profiles <b>SRK-R1</b> and <b>SATNAV-R1</b> .                                                        |
| Imported UGTS source   | 19 physical PDF pages: three front-matter pages plus 16 numbered main/appendix pages.                                 |
| Kernel delivered       | Literal word-feedback recurrence plus retained FP64 satnav; CPU and CUDA source.                                      |
| Navigation computation | Three ECEF position coordinates and one receiver-clock bias per independent epoch.                                    |
| UGTS integration       | Source-style support/compatibility/guard handoff, local chart, time/winding, exact key codecs and lineage records.    |
| Execution record       | Current subversion results appear in the SRK-R1 validation section; previous release evidence is retained separately. |
| On-device status       | Recorded from the actual local device in this subversion; see the current validation record.                          |

Buildable source and reproducible commands are included. The native recurrence records every state transition. The satnav laptop review script compares both input paths with CPU/Python and records Compute Sanitizer evidence.

**Source keys.** U is the attached *UGTS-KC 3.6.2 SCLP* PDF. References to U use its printed page numbers; add three for the physical PDF page. A is the retained 3.6.1 M2 specification/code history in this project. G1–G6 are researched primary GNSS/geodesy references; H1–H4 are NVIDIA hardware/API references. N01–N10 are explicit decisions in this specialization.

The earlier UGTS software archive and the prior aTOMos release archive were not mounted in this run. Their reported test totals are not adopted as tests of the new kernel. The earlier release records direct reading of U; this subversion inherits its transcribed equations and provenance. The original U and OTAN2 PDFs are not mounted here. [U, pp. 1–16]

## The two executable data flows

### **SRK-R1: literal self-reference**

Editable equations + initial state and masks → exact Boolean truth tables → old-state ASA/NA and J/K evaluation → committed next state → the next recurrence step. CPU and CUDA emit every intermediate; direct equation replay verifies the complete trace and identifies observed fixed points or cycles.

### **Prepared GNSS input**

Receiver code observations + additive model corrections + satellite positions at emission expressed in reception-time ECEF axes + per-observation uncertainty model.

### **aTOMos word plane**

Ready/present support → ASA → NA → boundary count and whole-word disposition.

### **Satnav kernel**

Selected channels → FP64 geometric ranges and residuals → weighted Givens QR → iterative position/clock solution → final residuals and formal covariance.

### **UGTS plane**

Position candidate → declared local supports → frame/time compatibility → conditional residual interval → computational event → observation record.

### **Indexed state**

ECEF and clock retained in full; ENU, log chart, separate hoop, full time/winding and both 64-bit keys are added to the lineage record. Literal OTAN2 remains a diagnostic.

This interpretation gives the GPU an actual repeated calculation: many independent receiver epochs, replay sessions or test cases. It does not require the display or ray-marching pipeline that U explicitly excludes. [U, pp. 1, 9–12]

## What satnav means in this release

The profile computes a receiver-position candidate from already decoded code ranges. A GNSS receiver or upstream software must supply the observations and modelling terms. There is no antenna interface, RF acquisition/tracking loop, signal synthesis, transmitter, route planner or vehicle-control loop in the package. No position is claimed to be measured from a GPU alone.

Position and clock are estimated independently at each epoch. Velocity filtering, carrier-phase ambiguity resolution, RTK/PPP and multi-constellation bias states are separate extensions, not hidden inside the four unknowns of this release.

## Contents

|                                                        |    |
|--------------------------------------------------------|----|
| Release and implementation record                      | 2  |
| The two executable data flows                          | 3  |
| 1 Literal self-reference: the completed recurrence     | 6  |
| 2 Actual equations are input data                      | 7  |
| 3 Fixed points, cycles and complete finite execution   | 8  |
| 4 Native execution and independent equation replay     | 9  |
| 5 Current subversion validation                        | 10 |
| 6 The imported UGTS and aTOMos contracts               | 11 |
| 7 Receiver state and source-symbol separation          | 12 |
| 8 The corrected-code observation model                 | 13 |
| 9 Input epochs, channels and aTOMos selection          | 14 |
| 10 The weighted positioning step                       | 15 |
| 11 Streaming QR, rank and formal covariance            | 16 |
| 12 Solver status and residual diagnostics              | 17 |
| 13 Local ENU and the UGTS log chart                    | 18 |
| 14 OTAN2 in satnav: preserve the observable            | 19 |
| 15 Paired supports and the canonical UGTS event        | 20 |
| 16 Finite cones and translational sweeps retained      | 21 |
| 17 Jitter intervals and the explicit source correction | 22 |
| 18 Two exact 64-bit key layouts                        | 23 |
| 19 Quantization, error and explicit origin handling    | 24 |
| 20 Linear time, winding and separate topology profiles | 25 |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>21 GPU kernel: one independent solve per thread</b>    | <b>26</b> |
| <b>22 Texture and global input paths</b>                  | <b>27</b> |
| <b>23 The RTX 5070 Ti Laptop target and VRAM plan</b>     | <b>28</b> |
| <b>24 Actual input and result files</b>                   | <b>29</b> |
| <b>25 Ordered event records and source lineage</b>        | <b>30</b> |
| <b>26 The retained deterministic satnav batch</b>         | <b>31</b> |
| <b>27 Independent results and error measures</b>          | <b>32</b> |
| <b>28 Prior release verification record</b>               | <b>33</b> |
| <b>29 Build, run and verify on the requested laptop</b>   | <b>34</b> |
| <b>30 Integration decisions and source issue register</b> | <b>35</b> |
| <b>31 Source and artifact inventory</b>                   | <b>36</b> |
| <b>Primary implementation references</b>                  | <b>37</b> |

## 1 Literal self-reference: the completed recurrence

Version 3.6.1.6 adds the executable profile **SRK-R1**. Its state is a 32-bit word  $q_t$  that occurs in its own next-state equation. The old release provided literal ASA/NA and J-K primitives but its observation adapter used a constant hold event. This subversion supplies an explicit feedback composition, editable equation input, CPU/CUDA execution and independent equation replay.

Let  $W = \{0, \dots, 2^{32} - 1\}$ , and let  $V, A, N, B, d \in W$  denote the present, ASA, NA, boundary and external drive words. All complements below are 32-bit complements. The initial state satisfies  $q_0 \wedge \neg V = 0$ . A trajectory supplies three Boolean expressions  $E_X, E_J, E_K$  and a requested number of steps  $T > 0$ . The complete transition, in evaluation order, is

$$x_t = E_X(q_t, d), \quad (1)$$

$$a_t = x_t \wedge A \wedge V, \quad (2)$$

$$n_t = a_t \wedge N, \quad (3)$$

$$h_t = \text{popcount}(n_t \wedge B), \quad (4)$$

$$y_t = \begin{cases} 0, & h_t > 0, \\ n_t, & h_t = 0, \end{cases} \quad (5)$$

$$J_t = E_J(q_t, y_t, d), \quad K_t = E_K(q_t, y_t, d), \quad (6)$$

$$q_{t+1} = [(J_t \wedge \neg q_t) \vee (\neg K_t \wedge q_t)] \wedge V. \quad (7)$$

The ASA tuple  $(a_t, n_t, h_t, y_t)$  is the retained whole-word rule. The bracketed part of (7) is the retained literal J-K equation. The supplied expression bindings and final projection to  $V$  are explicit SRK-R1 definitions. For a single active bit they give hold, set, reset and toggle for  $(J, K)$  equal to  $(0, 0), (1, 0), (0, 1), (1, 1)$  respectively.

### Self-reference is an executable dependency

$q_t \longrightarrow x_t \longrightarrow (a_t, n_t, h_t, y_t) \longrightarrow (J_t, K_t) \longrightarrow q_{t+1}$ , with  $q_t$  also read directly by both J/K expressions and by the final update. Every right-hand side reads the same old state. Only the final line commits the next state.

Boundary absorption sets the whole output word  $y_t$  to zero. It does not silently replace the user's J/K expressions with hold: those expressions still evaluate with that zero word. Thus the equations determine the transition even in an absorbed step. Numerical GNSS residual classifications do not gate SRK-R1.

## 2 Actual equations are input data

The editable input is `examples/self_reference/equations.json`. The root declares version 3.6.1.6, profile SRK-R1, optional default equations, and a trajectories array. Each trajectory supplies an ID, initial state, drive, four masks and step count; it may override the equations. For example:

```
{
  "version": "3.6.1.6", "profile": "SRK-R1",
  "equations": {"x": "q | d", "j": "y", "k": "y"},
  "trajectories": [{
    "id": 1, "q0": 0, "drive": 15, "present": 15,
    "asa_mask": 15, "na_mask": 15, "boundary_mask": 0,
    "steps": 6
  }]
}
```

The expression vocabulary is `&`, `|`, `^`, `~`, parentheses, the names `q`, `d`, and, for `J/K`, `y`. The literal `0` means the zero word; `1` means the all-ones word. Decimal state and mask fields remain ordinary unsigned integers. Operator precedence is complement, AND, XOR, OR; parentheses can specify every grouping explicitly.

The expression trees define an exact finite Boolean algebra. They are parsed and evaluated as that algebra; they are not passed to a general-purpose evaluator. Each expression is applied independently to each bit. Whole-word absorption couples the bits through  $h_t$ . Arbitrary cross-bit arithmetic, shift operators and continuous-state formulae would require a separately defined extension.

### 2.1 Exact truth-table compilation

For bits  $q_i, y_i, d_i \in \{0, 1\}$ , the compiler creates

$$L_X = \sum_{u,v \in \{0,1\}} E_X(u,v) 2^{2u+v}, \quad (8)$$

$$L_J = \sum_{u,v,w \in \{0,1\}} E_J(u,v,w) 2^{4u+2v+w}, \quad (9)$$

$$L_K = \sum_{u,v,w \in \{0,1\}} E_K(u,v,w) 2^{4u+2v+w}. \quad (10)$$

The CPU/CUDA kernel reads the corresponding table bit. For the example above,  $(L_X, L_J, L_K) = (14, 204, 204)$ . For the latch with  $K = 0$ , the tuple is  $(14, 204, 0)$ . Every allowed expression has a truth table on these inputs, and every Boolean function on these inputs is expressible in this vocabulary. The compilation is exact: each possible input assignment is evaluated, and the native table read retrieves that assignment's value.

### 3 Fixed points, cycles and complete finite execution

During one trajectory, equations, drive and masks are constant. They define a deterministic function  $F : Q_V \rightarrow Q_V$ , where

$$Q_V = \{q \in W : q \wedge \neg V = 0\}, \quad |Q_V| = 2^{\text{popcount}(V)}, \quad q_{t+1} = F(q_t). \quad (11)$$

**Proposition 3.1** (Closure and eventual periodicity). *Every computed state belongs to  $Q_V$ . Every infinite orbit of this fixed-input transition is eventually periodic. A fixed point is a cycle of length one.*

*Proof.* The input requires  $q_0 \in Q_V$  and the final AND with  $V$  places every successor in  $Q_V$ . Among  $|Q_V| + 1$  consecutive states at least two are equal. If  $q_i = q_j$  for  $i < j$ , determinism gives  $q_{i+k} = q_{j+k}$  for every  $k \geq 0$ . This proves periodicity after the first repeated state; it does not assert that the period is one.  $\square$

The implementation emits every requested step, including those after a repeat. Independent replay records a fixed point, a cycle, or no repeat observed within the requested budget. The last case is finite evidence only; it does not imply divergence or the absence of a later repeat. With 32 active bits the general state bound is  $2^{32}$ , so this theorem alone gives no short runtime bound.

#### 3.1 Worked exact trajectories

Set  $V = A = N = d = 15$ ,  $B = 0$ ,  $q_0 = 0$ , and  $E_X = q \vee d$ ,  $E_J = y$ . For the latch  $E_K = 0$ ,  $x = a = n = y = 15$  and

$$q_{t+1} = y \vee q_t = 15, \quad (q_0, q_1, q_2, \dots) = (0, 15, 15, \dots). \quad (12)$$

For toggle  $E_K = y$ ,  $J = K = 15$  and

$$q_{t+1} = q_t \oplus 15, \quad (q_0, q_1, q_2, q_3, \dots) = (0, 15, 0, 15, \dots). \quad (13)$$

With  $B = 8$  and  $E_J = E_K = y$ ,  $h = 1$  absorbs all of  $y$  and the state holds. With the same absorbed word but  $E_J = d$ ,  $E_K = 0$ , the state becomes 15. These outcomes follow directly from the supplied equations and expose the difference between output absorption and state reset.

Synchronous evaluation matters. For one active bit with  $E_J = \neg q$  and  $E_K = q$ , the result is toggle. Both expressions use  $q_t$ , never a partially updated state. State projection preserves inactive bits as zero, including when an input expression uses the all-ones constant.

## 4 Native execution and independent equation replay

The shared `include/self_reference.hpp` implements the truth-table and literal transition operations. The CPU walks each trajectory in step order. CUDA assigns one thread to each independently seeded trajectory, then executes its recurrence sequentially within that thread. No ordering between independent trajectories is needed; different trajectories never share a mutable state.

The compilation interface is a decimal CSV with columns

```
trajectory_id,q0,drive,present,asa_mask,na_mask,boundary_mask,
x_lut,j_lut,k_lut,steps
```

The header is physically one line. Every trace row retains

```
trajectory_id,step,before,drive,x,asa,na,hits,output,j,k,after
```

The native run also records backend, device, timing and trace lengths. The Python reference evaluates the original expression trees on whole integer words; it does not use the compiled truth tables as its transition oracle. It compares every trace field, row count and trajectory ordering, and records equation/input provenance. This cross-check can detect table compilation, transition and output corruption; it is finite execution evidence rather than a machine-checked proof.

### 4.1 Reproduce the equation-input run

From the package directory, build CPU and run:

```
cmake -S . -B build_cpu
cmake --build build_cpu --config Release --parallel 2
python tools/self_reference.py \
  --input examples/self_reference/equations.json \
  --out results/selfref_local \
  --binary build_cpu/Release/self_reference.exe --backend cpu
```

For single-configuration systems the binary is `build_cpu/self_reference`. For CUDA, enable `SATNAV_ENABLE_CUDA=ON`, choose the GPU binary and use `-backend cuda`. Each run uses a new output directory. The JSON expressions are editable without recompiling C++ or CUDA; the compiler regenerates their tables.

### 4.2 Relation to the existing SATNAV and UGTS equations

SATNAV-R1 retains its corrected-code equations, QR solve, OTAN2 source ratio, both 64-bit codecs and full coordinate/time records. Its observation adapter's hold event remains that profile's explicit choice. SRK-R1 is the new formal word state-feedback computation and runs without a GNSS residual admission condition. The new bindings are identified as definitions of this release. The unavailable earlier M2 circle-plus encoder and physical eigenmatrix are not asserted to have been reconstructed from the surviving primitives.

## 5 Current subversion validation

All entries here concern the executable 3.6.1.6 source on the current Windows machine. The 3.6.1.5 preparation record is preserved separately under `results_3_6_1_5/`. Finite comparisons check the requested trajectories; the closure and eventual-periodicity argument is given separately above.

| Executed check                        | Result                                                        |
|---------------------------------------|---------------------------------------------------------------|
| CPU CMake build and CTest             | Passed, 2 test programs                                       |
| Retained SATNAV C++ checks            | 105,559 passed                                                |
| New literal feedback C++ checks       | 5,060,172 passed                                              |
| Python test methods                   | 31 passed                                                     |
| Native command-line scenarios         | 25 feedback + 11 SATNAV passed                                |
| Editable six-trajectory example       | 34 transitions, 408 exact field comparisons per backend       |
| 257-trajectory stress input           | 4,225 transitions, 50,700 exact field comparisons per backend |
| CPU and CUDA equation replay          | Both passed sample and stress inputs                          |
| Feedback GPU Compute Sanitizer        | Zero memory errors; sanitized trace replay passed             |
| Retained SATNAV independent reference | 256 epochs, 4,828 comparisons passed                          |
| SATNAV CUDA texture/global paths      | Both passed independent comparisons and memcheck              |
| CPU sanitizers in this Windows run    | Not run; earlier Linux evidence archived                      |

### 5.1 Recorded toolchain and device

MSVC 19.44.35221.0, Visual Studio 2022 BuildTools; NVIDIA CUDA compiler 12.8.61; NVIDIA GeForce RTX 5070 Ti Laptop GPU, compute capability 12.0, driver 591.59. The source targets `sm_120` and `compute_120`. Device, runtime, allocation sizes and kernel-only times are retained in each native run record. The short external build directories used on this machine avoid Windows compiler scratch-path limits caused by the long workspace path.

The stress input uses seed 3616, varying 1–32 steps per trajectory, full 32-bit operands and 257 trajectories so that the 128-thread CUDA blocks include a partial final block. Direct AST replay checks all 12 trace columns. The Python suite also rejects deliberate mutations of every trace column, missing/extra rows and reordered rows. The C++ checks include independent bit oracles and exhaustive small domains.

### 5.2 Evidence and delivered implementation

`results/validation_status.json` indexes current evidence. The exact equation JSON, compiled CSV, complete native/reference traces and their hashes are retained. Windows CPU and CUDA executables are included under `bin/windows/`; CUDA execution uses the installed NVIDIA runtime and driver. No measured speedup is inferred from these small fixtures. The PDF is built with Tectonic 0.17.0 and visually checked after rendering. Source and delivered artifacts are listed in the regenerated SHA-256 manifest.

## 6 The imported UGTS and aTOMos contracts

The supplied prototype defines a finite, query-first state-and-relation package. It includes cone/-sphere relations, a bounded translational sweep, log-polar metric and kinematics, explicit time and winding, separate topology profiles, finite grammar, 64-bit key codecs and audited uncertainty/memory claims. Its authoritative handoff is

support  $\rightarrow$  compatibility  $\rightarrow$  guard  $\rightarrow$  verified event  $\rightarrow$  transition  $\rightarrow$  lineage. (14)

This order is retained in the downstream satnav event adapter. [U, pp. 1, 9–12]

### 6.1 What U does not define

U contains no satellite ephemeris propagation, receiver clock observation equation, code-delay correction model, geodetic datum or GNSS position estimator. These are new SATNAV-R1 definitions, grounded in primary GNSS references rather than attributed to the earlier prototype. [N01]

The base aTOMos contracts retained here are the four-field ASA word rule, explicit JK bit semantics, source-ratio OTAN2, typed unavailable values, immutable input snapshots and ordered result/provenance records. The new matrix in the receiver solve is the explicit *GNSS design matrix*. It is not a silently completed physical beamforming/eigenmatrix from an earlier profile. [A, retained M2 project contract]

### 6.2 Source figures and key table

U's Figure 4 on printed p. 5 keeps the half-turn state bundle distinct from the orientation-reversing Klein quotient. Its Table 1 and Figure 6 on printed p. 8 keep contiguous packing distinct from MSB-round-robin Morton interleaving. Figure 8 on printed p. 10 returns the result to the canonical event sequence. All three distinctions survive the specialization.

The complete earlier 50-operator catalog remains a source catalog, not a claim that every operator is necessary for positioning. The delivered helpers implement the subset selected below. A source mechanism omitted from the default satnav path is identified rather than repurposed without a declared meaning.

## 7 Receiver state and source-symbol separation

The authoritative positioning state is

$$z_e = (x_e, y_e, z_e, b_e) \in \mathbb{R}^4, \quad b_e = c \delta t_e, \quad (15)$$

where the first three components are ECEF metres and the fourth is receiver clock bias in metres. Clock seconds are  $b_e/c$  with  $c = 299792458$  m/s in this profile. This is the standard code-positioning unknown set. [G1]

| Quantity                    | Role                     | Representation                                                      |
|-----------------------------|--------------------------|---------------------------------------------------------------------|
| $r$                         | Receiver location        | Three unquantized ECEF doubles.                                     |
| $b$                         | Receiver clock bias      | Metres, not a branch bit.                                           |
| $P_i$                       | Code observation         | Metres, after the explicitly recorded correction convention.        |
| $\rho$                      | Local log-radius         | UGTS chart quantity, not the code range.                            |
| $\theta$                    | Local angular coordinate | $\text{atan2}(N, E)$ in a fixed ENU chart.                          |
| $\phi_h$                    | UGTS hoop phase          | Separate supplied periodic field; not silently geographic latitude. |
| $\varphi_g, \lambda_g, h_g$ | Geodetic coordinates     | Latitude, longitude and ellipsoidal height.                         |
| $X, w$                      | Tick and winding         | Full integer time identity retained beside its modular key field.   |
| $K_C, K_M$                  | Quantized addresses      | Two distinct 64-bit codecs, not complete navigation states.         |

The source's lower-case hoop  $\phi$  is retained as  $\phi_h$  to avoid colliding with geodetic latitude. The earlier aTOMos phase alias is resolved through a named adapter: OTAN2 receives increments of local  $\rho, \theta$ , while the key retains its separate UGTS hoop field. No 3D coordinate is silently removed. [U, p. 2][N02]

The authoritative records also include epoch identity, time scale, satellite/channel mapping, input/product provenance, statuses and uncertainty assumptions. All fit results preserve the full ECEF estimate and clock value outside the compact key.

## 8 The corrected-code observation model

A decoded code measurement can be modelled as a geometric range, receiver/satellite clock terms and selected propagation/instrumental delays. SATNAV-R1 accepts an upstream additive correction  $a_i$  and defines

$$\tilde{P}_i = P_i + a_i = \|s_i - r\| + b + \varepsilon_i. \quad (16)$$

This is the reduced standard code-positioning model after the declared upstream terms are accounted for. [G1, G4]

For the convention

$$P_i = R_i + b - c\delta t_i^{\text{sat}} + T_i + I_i + B_i + \varepsilon_i, \quad (17)$$

the matching additive correction is

$$a_i = c\delta t_i^{\text{sat}} - T_i - I_i - B_i. \quad (18)$$

The adapter must define the exact code, satellite clock convention, relativistic clock terms and hardware/group biases it uses. The kernel adds the supplied term *once*. It does not repeat an already applied correction or invent a missing ionosphere/troposphere model.

### 8.1 Satellite coordinates and Earth rotation

$s_i$  is the satellite position at signal emission expressed in the Earth-fixed axes associated with reception. The upstream stage must perform the necessary transmit-time and frame handling consistently. A satellite coordinate computed in emission-time ECEF axes cannot simply be treated as already in the reception axes. [G2]

The solver keeps these supplied positions and corrections fixed during its four-state iteration. The profile is therefore an adapter boundary for prepared code data, not a complete broadcast-navigation decoder. Raw receiver or RINEX values require an upstream modelling adapter before using the named CSV contract. [N01]

### 8.2 One clock system

There is exactly one receiver-clock unknown. Unaligned measurements from multiple GNSS time systems generally need additional bias treatment; this release does not estimate a hidden inter-system offset. Use compatible code/clock data, or supply and document an upstream alignment under a separate profile.

## 9 Input epochs, channels and aTOMos selection

Each independent epoch binds its receive-time label, approximate initial state and 32-bit channel masks. Each observation row points to that epoch and names a unique channel in  $0, \dots, 31$ . Channel IDs, not row order, define mask bits.

Let  $V$  contain present channels and  $X$  contain channels marked ready. The retained word selection is

$$a = X \wedge M_{\text{ASA}} \wedge V, \quad (19)$$

$$n = a \wedge M_{\text{NA}}, \quad (20)$$

$$h = \text{popcount}(n \wedge M_{\partial}), \quad (21)$$

$$y = \begin{cases} 0, & h > 0, \\ n, & h = 0. \end{cases} \quad (22)$$

The solver uses precisely the channels selected by  $y$ . These four fields are emitted unchanged in the result. The default ASA/NA masks are all ones and the default boundary is zero. [A, retained M2 project contract] [N04]

### 9.1 Whole-word behavior remains literal

A boundary bit intersecting the selected word absorbs the entire epoch group. It does not mean “drop only this satellite.” To explicitly omit an individual observation, use a selector bit, a not-ready flag or an upstream policy with its own provenance. Replacing the boundary behavior with per-bit clearing would change the base engine.

A remaining support of fewer than four channels returns T00\_FEW. Four channels must still provide a numerically independent design matrix. A full bit count cannot prove observability by itself. The new GNSS solver checks the resulting matrix rather than treating POPCNT as a complete position solution. [G1][N03]

### 9.2 Timeline and schema consistency

An epoch identifier binds rows from one receiver event. The time column is full unwrapped GPST seconds under the supplied upstream convention; modular UGTS time is added later. Duplicate epoch IDs, duplicate channels or duplicate satellite IDs within an epoch are errors. Correcting a false upstream time label is not something a CSV parser can infer from equal strings.

The current CSV reader uses finite bounded numeric fields and explicit nonnegative IDs. Physical correction provenance belongs to the input profile; metadata consistency is not a physical synchronization measurement.

## 10 The weighted positioning step

For current estimate  $z^{(k)} = (r^{(k)}, b^{(k)})$ , define

$$R_i^{(k)} = \|r^{(k)} - s_i\|, \quad (23)$$

$$v_i^{(k)} = \tilde{P}_i - R_i^{(k)} - b^{(k)}, \quad (24)$$

$$H_i^{(k)} = \left[ \frac{x^{(k)} - s_{ix}}{R_i^{(k)}}, \frac{y^{(k)} - s_{iy}}{R_i^{(k)}}, \frac{z^{(k)} - s_{iz}}{R_i^{(k)}}, 1 \right]. \quad (25)$$

Linearization gives  $H\Delta z \simeq v$ . With declared independent code standard deviations  $\sigma_i > 0$ , whiten the rows as

$$A_i = H_i/\sigma_i, \quad d_i = v_i/\sigma_i, \quad \Delta\hat{z} = \arg \min_{\Delta} \|A\Delta - d\|_2^2. \quad (26)$$

Then update  $z^{(k+1)} = z^{(k)} + \Delta\hat{z}$ . The nonlinear model and iterative linearization follow the code-positioning references; the QR implementation is the explicit numerical choice of this specialization. [G1, G4][N03]

### 10.1 No normal-equation update

The C++/CUDA implementation uses streaming Givens QR on the whitened rows. It does not form and invert  $A^T A$  for the update. This avoids squaring the design matrix's condition number as part of the solution method. The independent Python reference uses a different Householder QR implementation on the full row matrix.

All four unknowns use metres, including the clock bias. A floating-point conversion to seconds is applied only to a displayed/consumer clock offset. This keeps a factor of the speed of light out of the fourth Jacobian column.

The default iteration stops when both position-increment norm and absolute clock increment are at most  $10^{-4}$  m, with at most 12 steps. These are numerical convergence controls, not a statement that real measurements yield 0.1 mm accuracy. The convergence domain depends on the input model and initial approximation.

## 11 Streaming QR, rank and formal covariance

The kernel maintains an upper-triangular  $4 \times 4$  matrix  $R$  and a transformed right-hand side  $z$ . Each whitened row is appended by rotations that eliminate its components against the current triangular diagonal.

For diagonal  $R_{jj}$  and current row component  $a_j$ , let

$$\gamma = \text{hypot}(R_{jj}, a_j), \quad c_j = R_{jj}/\gamma, \quad s_j = a_j/\gamma. \quad (27)$$

The same rotation is applied to the remaining row and right-hand-side entries. A zero pair needs no rotation. The result is solved by back substitution.

### 11.1 The implemented rank screen

The retained check is

$$|R_{jj}| > \tau \max_k |R_{kk}| \quad \text{for all } j, \quad \tau = 10^{-10}. \quad (28)$$

This is a declared unpivoted triangular-diagonal screen. It detects the tested rank-deficient cases, but is not an SVD condition-number certificate or a proof of well-conditioned geometry for all possible inputs. A failed screen produces `RANK_DEFICIENT`, not a coordinate filled with zeros. [N03]

### 11.2 Covariance is tied to the assumed sigmas

After convergence the Jacobian is rebuilt at the final estimate. From  $R^{-1}$ , compute the diagonal of

$$P = (H^T W H)^{-1} = R^{-1} R^{-T}, \quad W = \text{diag}(\sigma_i^{-2}). \quad (29)$$

The implementation obtains this from triangular solves, not a dense matrix inverse of  $H^T W H$ . It returns coordinate/clock variances in squared metres. This is formal uncertainty under the declared local linear and noise model. [G5]

It is not measured error against truth, and not a guaranteed bound on faulty or mis-modelled measurements. The independent fixture comparisons check both estimates and these formal diagonal values; true-error metrics are separately computed only because the artificial fixture has a known generating position.

## 12 Solver status and residual diagnostics

Convergence, residual fit and the later UGTS event are separate fields.

| Solver status   | Meaning                                                               |
|-----------------|-----------------------------------------------------------------------|
| CONVERGED       | Numerical increment thresholds reached with valid final calculations. |
| MASK_ABSORBED   | Selected word intersected the retained whole-word boundary.           |
| TOO_FEW         | Fewer than four selected observations.                                |
| RANK_DEFICIENT  | Triangular rank/covariance check failed.                              |
| ITERATION_LIMIT | The selected number of iterations did not satisfy convergence.        |
| NUMERIC_FAILURE | Invalid range, nonfinite result or unusable numeric intermediate.     |

For a converged solution, preserve every selected post-fit residual  $r_i = \tilde{P}_i - \|s_i - \hat{r}\| - \hat{b}$ , and report

$$\text{RMS} = \sqrt{\frac{1}{m} \sum_i r_i^2}, \quad S = \sum_i (r_i/\sigma_i)^2, \quad Z_{\max} = \max_i |r_i/\sigma_i|. \quad (30)$$

The default comparison  $Z_{\max} \leq 6$  is a named residual budget. It is not a leverage-normalized test or a claimed false-alarm probability. The solver does not automatically discard a channel and retry until this flag passes. [N09]

Exactly four observations give NO\_REDUNDANCY, even if their residuals are small. For more observations, the flag is WITHIN\_RESIDUAL\_BUDGET or RESIDUAL\_ALERT. A single gross-outlier fixture converges numerically but produces the latter alert; the data record retains its large actual position error.

RAIM/ARAIM, HPL/VPL, continuity risk, certified spoof detection and navigation-action authority are not supplied by these flags. A valid solution record can be useful for replay and analysis without representing those additional guarantees.

### 13 Local ENU and the UGTS log chart

The receiver solution remains three-dimensional. For a declared geodetic anchor  $(\varphi_0, \lambda_0, h_0)$ , convert ECEF differences to East/North/Up using

$$R_{ENU} = \begin{bmatrix} -\sin \lambda_0 & \cos \lambda_0 & 0 \\ -\sin \varphi_0 \cos \lambda_0 & -\sin \varphi_0 \sin \lambda_0 & \cos \varphi_0 \\ \cos \varphi_0 \cos \lambda_0 & \cos \varphi_0 \sin \lambda_0 & \sin \varphi_0 \end{bmatrix}. \quad (31)$$

Then  $(E, N, U)^T = R_{ENU}(\mathbf{r} - \mathbf{r}_0)$ . The transform is an explicit GNSS/geodesy addition, separate from the source's abstract local chart. [G3]

With horizontal radius  $r_h = \sqrt{E^2 + N^2}$ , use

$$\rho = \ln(r_h/r_0), \quad \theta = \text{atan2}(N, E). \quad (32)$$

Up remains a separate value. A declared core radius returns `origin_core` instead of evaluating a singular logarithm. Out-of-range  $\rho$  is not automatically wrapped into a different geographic position. [U, pp. 3–4][N02]

#### 13.1 Metric and finite steps carried from U

For the planar chart,

$$ds^2 = r_h^2(d\rho^2 + d\theta^2), \quad \Delta r_h = r_h(e^{\Delta\rho} - 1), \quad (33)$$

$$J = r_h \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (34)$$

These identities explain the local representation; the new satellite-range solve still uses full ECEF coordinates. Quantizing this chart does not improve the original measurement precision or create a third coordinate. [U, p. 4]

The geodetic utility uses the declared WGS84 ellipsoid constants  $a = 6378137$  m and  $1/f = 298.257223563$ . Its height is ellipsoidal height, not an unprovided geoid-corrected height above mean sea level. The reference tests its conversions and inverse relation over a finite set of points. [G6]

## 14 OTAN2 in satnav: preserve the observable

The inherited literal operator remains

$$\text{otan2}_s(\Delta\theta, \Delta\rho) = \arctan(\Delta\theta/\Delta\rho), \quad \Delta\rho \neq 0, \quad (35)$$

with a separately supplied reference-axis subtraction when selected. This is a *log-chart motion slope*. It is not used to solve pseudoranges, and is not relabeled as the receiver's geographic heading or satellite azimuth. [A, retained M2 project contract]

The passive adapter compares successive defined local chart records in the same frame. It uses an explicitly named principal-difference policy for  $\Delta\theta$ ; that policy does not reconstruct missed revolutions. Full linear time and the chosen phase convention are stored with the observation.

### 14.1 Missing direction remains a status

A first observation has no previous chart increment. A zero pair has no direction. A pure angular increment has an undefined literal ratio. The adapter records `first_observation`, `zero_increment` or `ratio_undefined` rather than inventing a perfect zero angle. A sequence with non-increasing time is identified instead of being silently reordered into a new physical history.

A change in the axis alone changes the raw residual by the opposite angle. A common positive scaling of both increments preserves the ratio, and simultaneous reversal also preserves the literal ratio while reversing the actual directed motion. These are inherited operator laws, not new navigation-sensor guarantees.

### 14.2 Literal J/K and the explicit feedback profile

The retained equation is

$$q^+ = (J \wedge \neg q) \vee (\neg K \wedge q). \quad (36)$$

SRK-R1 binds J and K to supplied expressions that read the old state and literal ASA output, then carries  $q^+$  into the next step. The full recurrence, expression input and execution semantics are given in Section 1.

The SATNAV-R1 observation adapter retains its explicit hold event  $J = K = 0$  as a compatibility profile. This choice does not restrict the separate SRK-R1 equation inputs. The original encoded circle-plus blend and predicate encoders are still not supplied in the mounted source, so the new feedback bindings are identified as new definitions rather than reconstructed earlier encoders.

## 15 Paired supports and the canonical UGTS event

U normalizes the two spherical supports as

$$f_L(x) = \|x - c_L\| - R_L, \quad f_R(x) = \|x - c_R\| - R_R, \quad (37)$$

with a profile selecting union, intersection or either-with-tag. The overlap flag is retained separately. [U, p. 3]

SATNAV-R1 uses the union of two *declared local processing domains* for its worked event adapter. With supplied deterministic position-radius allowance  $u_p$ , its per-sphere margin is

$$m_j = R_j - \|x - c_j\| - u_p. \quad (38)$$

Admission means at least one margin is nonnegative. This is a geometric/query policy for the supplied local frame, not a claim that the true receiver error is bounded by  $u_p$ . Its origin is explicitly recorded. [N09]

The corresponding handoff is implemented in order:

| UGTS stage     | Satnav realization                                                                      |
|----------------|-----------------------------------------------------------------------------------------|
| Support        | Candidate lies within at least one declared local support under the supplied allowance. |
| Compatibility  | Fixed frame and increasing unwrapped time; supported estimator-clock convention.        |
| Guard          | Residual interval compared with a named metre-domain budget.                            |
| Verified event | Candidate satisfies those computational/model conditions with residual redundancy.      |
| Transition     | Publish an observation record only. No external control command.                        |
| Lineage        | Source/profile IDs, full time, source keys and the preceding record digest.             |

For the demo,  $u_p = 5$  m, the two support radii are 10,000 m, and centres are (0, 0, 0) and (100, 100, 0) ENU metres. These are labelled worked parameters, not values extracted from the attached prototype or a real site survey.

The phrase POSITION\_CANDIDATE\_VERIFIED means verified under this declared computational handoff. It does not supply certified GNSS integrity or prove the physical correctness of the upstream observations.

## 16 Finite cones and translational sweeps retained

For the source's slant length  $T > 0$  and half-angle  $0 < \alpha_c < \pi/2$ ,

$$h = T \cos \alpha_c, \quad R = T \sin \alpha_c. \quad (39)$$

With apex  $c$ , unit axis  $a$  and query  $x$ , set

$$z = a \cdot (x - c), \quad q = \|(x - c) - za\|. \quad (40)$$

The exact primitive relation is the signed meridian distance to the triangle  $(-R, h), (R, h), (0, 0)$ , with negative interior and zero boundary. The implementation computes distances to the three boundary segments and uses the source's inside test  $0 \leq z \leq h, q \leq z \tan \alpha_c$ . [U, pp. 2–3]

These are optional UGTS query helpers in the archive. A finite cone is not silently substituted for the satellite pseudorange sphere or a physical antenna response. The default receiver solve uses the actual distance-to-satellite equations already stated.

For a fixed-orientation cone translated along a segment of length  $L$ ,  $n \geq 2$  uniform samples and sample minimum  $m_n$ , retain

$$m_n - \frac{L}{2(n-1)} \leq \inf_{u \in [0,1]} d_C(x - s(u)) \leq m_n. \quad (41)$$

The source derives this using the one-Lipschitz distance response to translation and the nearest sample spacing. It is not a general exact SDF for arbitrary rotated sweeps. [U, p. 3]

### 16.1 Numerical and nesting distinction

The helper evaluates this analytic interval in ordinary floating arithmetic. It does not implement directed rounding or a proven machine-error enclosure. The interval's *width* decreases with sample spacing. Intervals from arbitrary nonnested sample grids need not be nested, so the implementation does not claim that stronger property. [N07]

The source's constraint-release and hinge-torque mechanisms remain separately typed geometric/mechanical profiles. Removing a constraint is not an automatic improvement to satellite observability, nor does the existence of a hinge model supply a physical receiver trajectory. They are not inserted into the default position estimator.

## 17 Jitter intervals and the explicit source correction

U defines  $f_j = f + \epsilon_j \sigma$  with  $\sigma \in \{-1, +1\}$ , then prints the equality

$$[f_j - \epsilon_j, f_j + \epsilon_j] = [f - \epsilon_j, f + \epsilon_j]. \quad (42)$$

That equality is generally false for nonzero  $\epsilon_j$ . This edition preserves its source location and explicitly separates the two interpretations rather than silently changing the supplied page. [U, p. 5][N06]

### 17.1 Two valid, different interval statements

The perturbation family obeys

$$f_j \in [f - \epsilon_j, f + \epsilon_j]. \quad (43)$$

If  $f_j$  is the available perturbed observation and  $|f_j - f| \leq \epsilon_j$ , then

$$f \in [f_j - \epsilon_j, f_j + \epsilon_j]. \quad (44)$$

Both are valid enclosure statements, but the interval centres differ. The helper returns both named intervals and a test verifies that they are not falsely equated. The source's example centred around  $f$  is treated as the perturbation-family interval.

No jitter is added to the GNSS code observations or to solved ECEF coordinates. The deterministic bit can be metadata or a separate query perturbation, not a method for manufacturing measurement precision or removing multipath.

### 17.2 Metre-domain guard in the event adapter

For the largest absolute post-fit residual  $m = \max_i |r_i|$  and a *supplied* absolute residual perturbation allowance  $u_r$ , form

$$[\max(0, m - u_r), m + u_r]. \quad (45)$$

A budget  $B$  is met when the upper endpoint is at most  $B$ , exceeded when the lower endpoint is greater than  $B$ , and otherwise unresolved. This is a conditional model classification. The demo uses  $u_r = 0.1$  m and  $B = 20$  m as stated fixture parameters. It does not identify nominal  $\sigma_i$  with a hard worst-case bound.

## 18 Two exact 64-bit key layouts

U's field widths and layouts are retained exactly:

| Field            | Bits | Contiguous range | Domain                       |
|------------------|------|------------------|------------------------------|
| Log-radius index | 20   | 63:44            | Declared $\rho \in [-20, 0]$ |
| Chart angle      | 18   | 43:26            | Periodic $[0, 2\pi)$         |
| Modular tick     | 14   | 25:12            | $0, \dots, 16383$            |
| Hoop phase       | 12   | 11:0             | Periodic $[0, 2\pi)$         |

The contiguous word is

$$K_C = (q_\rho \ll 44) \vee (q_\theta \ll 26) \vee (q_X \ll 12) \vee q_{\phi_h}. \quad (46)$$

The Morton word emits source bits in MSB round-robin order

$$\rho_{19}, \theta_{17}, X_{13}, \phi_{11}, \rho_{18}, \theta_{16}, X_{12}, \phi_{10}, \dots \quad (47)$$

until each source field is exhausted. They are intentionally different encodings. [U, pp. 7–8]

### 18.1 The source's exact golden pair

For (949111, 0, 1920, 227), the new tests recover

```
contiguous: e7b77000007800e3
Morton:    88823bb88099128b
```

Both decode to the same quantized tuple. Random round trips and prefix-refinement bounds are tested separately from the new positioning arithmetic.

### 18.2 Address identity is not state identity

The 64-bit key has exactly  $2^{64}$  possible words. A key does not encode arbitrary clock bias, full linear time, Up, formal covariance, correction products or lineage. Those remain separate full fields in the event record. The 12-bit field is the named hoop phase, not a simultaneously hidden arbitrary matrix and grammar payload. [U, pp. 8–9][N05]

The address construction follows the position estimate; it is not used to quantize satellite coordinates or code ranges before the receiver solve. A matching key is an index match, not proof that two entire navigation events are identical.

## 19 Quantization, error and explicit origin handling

SATNAV-R1 declares nearest-node radial quantization over  $[-20, 0]$ :

$$h_\rho = \frac{20}{2^{20} - 1}, \quad q_\rho = \text{round}_+ \left( \frac{\rho + 20}{20} (2^{20} - 1) \right). \quad (48)$$

Angles use nearest periodic nodes with upward half ties, while the source's bit widths and range remain unchanged. This tie/quantization choice is an explicit specialization, not an unquoted algorithm recovered from an unavailable source ZIP. [N05]

For an angular width  $b$ ,  $h_\theta = 2\pi/2^b$ . A nearest-node quantization gives  $|\delta\rho| \leq h_\rho/2$  and circular  $|\delta\theta| \leq h_\theta/2$  within the chosen range. Using the exact source radial step, a conservative planar error bound is

$$\|\hat{x} - x\| \leq r_h(e^{h_\rho/2} - 1) + 2r_h e^{h_\rho/2} \sin(h_\theta/4). \quad (49)$$

It follows from adding a radial displacement and an angular chord at the enlarged radius. The bound scales with radius; log packing is not uniform physical resolution. [U, pp. 4, 8–9]

### 19.1 Default diagnostic chart

The demo chooses  $r_0 = 100000$  m and a 0.1 m origin core. It keys only values with  $-20 \leq \rho \leq 0$ . These choices define an indexed diagnostic domain, not the receiver's positioning accuracy or a geographic route. Out-of-range or core values keep the full position and a chart status without wrapping them into a new position.

### 19.2 No equal-information compression claim

U explicitly audits the ratios 192/64, 32/1 and 512/12 as nominal width ratios between records with different semantics. This specialization does not reintroduce a claim that one address is equivalent to all original floating information. [U, pp. 8–11]

The packed key is useful for lookup and local indexing after the relevant state exists. Position estimation, corrected observations, uncertainty and event verification remain real computational and storage costs.

## 20 Linear time, winding and separate topology profiles

For reference tick  $X_0$ , period  $P > 0$  and full linear tick  $X$ ,

$$u = \frac{X - X_0}{P}, \quad w = \lfloor u \rfloor, \quad \psi = u - w \in [0, 1). \quad (50)$$

The source's example  $X = 1920$ ,  $X_0 = 1135$ ,  $P = 256$  yields  $w = 3$  and  $\psi = 0.06640625$ . The new helper reproduces that value, including floor-division behavior for negative ticks. [U, p. 5]

The satnav key stores only the 14-bit tick residue, while the event retains full tick, GPST coordinate, origin/period and winding. A period of 16384 one-second ticks repeats its residue after 4 h 33 min 4 s. It does not identify events across those repetitions without the additional time/lineage fields.

### 20.1 The source half-turn is not the reflective quotient

U's literal odd radial-wrap bundle applies

$$(\rho, \theta, \phi_h, o) \mapsto (\rho', \theta + \pi, -\phi_h, -o). \quad (51)$$

Its separately named reflective Klein profile applies

$$(\rho, \theta, \phi_h, o) \mapsto (\rho', \pi - \theta, -\phi_h, -o). \quad (52)$$

The second reverses angular orientation; the first is a rotation on the base circle with a changed internal state. The archive retains independent helpers and tests, matching the distinction in U's Figure 4. [U, pp. 5–6]

Neither rule is applied to the GNSS receiver's ECEF position, a satellite orbit or the physical Earth. A local index experiment may select either profile explicitly; it does not make a geographical discontinuity an actual displacement. [N08]

The source's finite grammar and radix refinement remain local state/index mechanisms. Their existence does not turn one receiver solution into an unbounded tree or prove that every possible trajectory is retrieved in constant time.

## 21 GPU kernel: one independent solve per thread

The kernel takes a bounded batch of immutable epoch metadata and a structure-of-arrays observation buffer. One thread evaluates one full independent four-state solution. Each thread owns its QR workspace, iterative estimate, status and residual record. There is no cross-epoch update or unordered reduction of position states.

```
const size_t e = size_t(blockIdx.x) * blockDim.x + threadIdx.x;
if (e < count) {
    output[e] = solve_epoch(epochs[e], observations, e, config);
}
```

The launch uses 128 threads per block. Repeated jobs can supply thousands of independent epochs; a single live low-rate receiver does not by itself provide enough parallel work to establish a GPU speed advantage. This kernel is a batch/replay specialization.

### 21.1 Numerical implementation

The actual shared implementation is in `include/satnav_core.hpp`. It performs the retained ASA arithmetic, row construction, QR update, convergence and final covariance/residual calculation. The CUDA translation unit invokes that implementation with its device reader. The CPU uses the same word and numerical function with a plain double reader.

An independent Python implementation uses separate matrix construction and Householder QR. It is not merely calling back into the shared C++ function. The two verification levels therefore distinguish device/host conformance from an additional numerical reference comparison.

### 21.2 Potential performance costs remain visible

Independent threads may finish in different iterations, producing normal warp imbalance. QR uses FP64 arithmetic and square roots; no tensor-core speed claim is made. The source keeps full precision where satellite-range subtraction requires it, rather than forcing coordinates into float textures or one-bit words.

The build disables fused contraction for the shared review profile and does not enable fast math. The acceptance checks use named tolerances, not a claim of bit-identical floating evaluation across every compiler and processor.

## 22 Texture and global input paths

For  $N$  independent epochs, channel  $c \in [0, 31]$  and observation field  $f \in [0, 5]$ , flatten the input as

$$\text{index}(e, c, f) = (6c + f)N + e. \quad (53)$$

At a fixed channel/field, neighboring epoch threads therefore read neighboring values. The six fields are satellite  $x, y, z$ , raw code, additive correction and nominal sigma.

### 22.1 Exact FP64 bit transport in the texture path

A CUDA linear texture resource uses an `int2` channel. Each texel carries the 64 raw bits of one double; the device reconstructs the value using the low and high 32-bit words. The read uses an integer `tex1Dfetch`; there is no interpolation, normalized-coordinate lookup or conversion of ranges into float values. [H4][N10]

```
const int2 bits = tex1Dfetch<int2>(texture, integer_index);
const double value = __hiloInt2Double(bits.y, bits.x);
```

The global path reads the same double buffer directly. Both paths use the same solver and output record. Equality of decoded inputs is a data-representation property; it does not guarantee a texture speedup for this access pattern.

### 22.2 Ownership and publication

Device buffers and events use local ownership wrappers. A texture object is destroyed before its backing buffer. Each launched epoch has one writer. The program checks CUDA allocation, copies, launch and completion before publishing downloaded result files. A failed device call is an error, not a CPU fallback labelled as a GPU success.

The first chunk receives an untimed warmup. Device events measure subsequent kernel intervals and their sum is reported. CSV parsing, allocations, texture construction, host packing, uploads, downloads and the Python event adapter are outside that sum. A CPU loop interval and a full application latency are therefore distinct quantities.

## 23 The RTX 5070 Ti Laptop target and VRAM plan

NVIDIA lists the **RTX 5070 Ti Laptop GPU** with a **12 GB GDDR7** memory configuration. NVIDIA's CUDA table lists the GeForce RTX 5070 Ti family at compute capability 12.0; the build selects `sm_120` and matching virtual code. CUDA 12.8 introduced compiler support for `SM_120`. These are supporting implementation facts, not equations supplied by the UGTS prototype. [H1–H3]

The program's `-probe` path records the actual device name, major/minor capability, driver/runtime versions, total/free memory and linear-texture limit. This edition's targeted binary reports a mismatching capability rather than assuming that a marketing label guarantees the compiled architecture.

### 23.1 Payload accounting

The executed CPU build reports 72 bytes per epoch metadata record and 376 bytes per result. The fixed observation representation is  $32 \cdot 6 \cdot 8 = 1536$  bytes per epoch, so the tested ABI payload is

$$M_{\text{payload}}(N) = N(72 + 376 + 1536) = 1984N \text{ bytes.} \quad (54)$$

The 256-epoch demo therefore uses 507,904 payload bytes. A maximum chunk of 65,536 epochs uses 124 MiB before a separate 64 MiB planning margin. These figures do not count the entire driver/context or all host copies.

Admission uses the actual free/total bytes, the user budget and reserve. The defaults are a 512 MiB budget and 1536 MiB total-device reserve. After checking that each subtraction is valid, capacity is bounded by

$$B_{\text{usable}} = \min(B_{\text{user}}, 0.7F, T - R_{\text{reserve}}) - 64 \text{ MiB.} \quad (55)$$

The integer implementation rounds the free-memory fraction downward, caps each chunk at 65,536 epochs and checks texture/index limits. Another process may consume memory after probing, so actual allocation errors still propagate.

The package does not fill all 12 GB to demonstrate activity. Useful payload depends on the batch; automatically leaving space for other laptop processes is a resource plan, not a claim of permanent texture-cache residency. No operating-system ring-0 driver or executable-code-in-texture mechanism is part of the CUDA interface.

## 24 Actual input and result files

The executable reads two CSV files with exact headers. The source-to-domain model is recorded separately in `profile.json`; prepared real data must supply its own correction/product provenance.

```
epochs.csv:
epoch_id,t_rx_gpst_s,x0_m,y0_m,z0_m,b0_m,
asa_mask,na_mask,boundary_mask

observations.csv:
epoch_id,channel,satellite_id,sx_rx_m,sy_rx_m,sz_rx_m,
code_m,add_correction_m,sigma_m,ready
```

The display above wraps the headers for readability; the actual files use one header line each. `docs/INPUT.md` defines their exact names, ranges and physical meaning. The reader rejects malformed headers, duplicate channels, nonfinite fields and invalid sigmas before passing data to a backend.

### 24.1 Results

`solutions.csv` retains one row per input epoch: source/time identity, solver and fit status, iteration count, channel count, all four ASA fields, position/clock, formal diagonal covariance and residual statistics. `residuals.csv` records each active post-fit channel when the solver converges.

For a failed solve, estimate fields may contain the starting guess or a partial iterate. The status is authoritative; those fields must not be published as a valid fix. Zero variances in an unevaluated result do not mean a perfectly known position. The parser and status record prevent that interpretation without hiding the original calculation state.

`run.json` records the backend, timing scope, selected numerical settings and actual device report where available. `ugts_events.jsonl` and `ugts_summary.json` are produced by the explicit post-processing step. They do not replace the original solution or residual files.

A new output directory is required for each run. Completed runs can be compared without silently overwriting earlier evidence. Corruption of a source file or a changed result is visible in the delivered integrity manifest or retained event chain.

## 25 Ordered event records and source lineage

A UGTS event binds the solved candidate, coordinate transforms, full time, optional keys, support-/guard record and previous digest. The source's ordering is retained, while the satnav-specific meanings are labelled explicitly.

```
{
  "version": "3.6.1.6",
  "quantity": "receiver_position_candidate",
  "epoch_id": 0,
  "solver_status": "CONVERGED",
  "ecef_m": ["three retained numeric components"],
  "clock_bias_m": "retained numeric clock bias",
  "enu_m": ["E", "N", "Up retained separately"],
  "chart": {"rho": "...", "theta": "...", "hoop_phi": "..."},
  "packing": {"contiguous_hex": "...", "morton_hex": "...",
    "linear_tick": 0, "winding": 0},
  "support": {}, "compatibility": {}, "guard": {},
  "event": "POSITION_CANDIDATE_VERIFIED",
  "transition": "publish_observation_only",
  "previous_hash": "...", "record_hash": "...
}
```

This is a structural sketch; the complete worked exports contain actual numeric values and statuses, not these explanatory strings.

The chain uses a domain-tagged SHA-256 of canonical local JSON and retains the prior head. It binds the emitted sequence to a retained reference. It is not a signature, a satellite-signal authentication primitive or proof that an upstream observation was physically true.

Author attribution is included as supplied: Tom Klootwijk, with the identifier and date in the release metadata. A full profile/source digest remains separate from the 64-bit coordinate key or a one-bit event field. This preserves the base engine's identity/state distinction and U's requirement that uncertainty and lineage survive packing. [A, retained M2 project contract] [U, pp. 8–10]

## 26 The retained deterministic satnav batch

The shipped demo has 256 artificial receiver epochs. The anchor is a declared WGS84 location at 52° latitude, 5° longitude and 100 m ellipsoidal height. Receiver points are generated from a local ENU path, with a known clock bias near 75,000 m. Satellite directions and ranges are constructed fixtures, not a downloaded constellation or an observation session from the user's laptop.

The first 128 cases are noiseless model data. The next ordinary cases add independent Gaussian fixture noise with their specified sigmas. The final six cases deliberately exercise failure and edge conditions. Corrections are labelled artificial additive test values, not an atmosphere or ephemeris model.

| Outcome                                 | Observed count / result |
|-----------------------------------------|-------------------------|
| Converged numerical epochs              | 253 of 256              |
| Too few selected satellites             | 1                       |
| Whole-word absorbed                     | 1                       |
| Rank-deficient geometry                 | 1                       |
| Converged within chosen residual budget | 251                     |
| Gross-outlier residual alert            | 1                       |
| Exactly four, no residual redundancy    | 1                       |

The exact-four and 32-channel cases verify those interface boundaries. The outlier case adds an artificial 200 m error to one code observation. It converges to an incorrect candidate with a large residual alert, demonstrating why convergence and navigation reliability are not the same field.

The geometry-support adapter then emits 251 computationally verified candidate records, three no-solution records and two unresolved candidates under its declared conditions. The exact-four case remains unresolved because no redundancy is present; the outlier remains unresolved because the chosen residual criteria are exceeded.

These are finite results of a known model. The demo is useful for reviewing signs, units, frame adapters and implementation agreement. It is not a real-world accuracy measurement or a GPU speed result.

## 27 Independent results and error measures

The C++ solve is compared against a separately written Python Householder solver. The comparison checks epoch identity, all mask fields, status/fit, position and clock, formal covariance and residual statistics, yielding **4,828 checked comparisons** over the 256 epochs.

| Cross-reference quantity              | Observed maximum                          |
|---------------------------------------|-------------------------------------------|
| Absolute state-component difference   | $1.210719347 \times 10^{-8} \text{ m}$    |
| Formal covariance diagonal difference | $6.394884622 \times 10^{-14} \text{ m}^2$ |

The comparison tolerance for each state component is  $10^{-4} \text{ m}$ . Small algorithmic agreement does not make the underlying physical inputs exact; it shows that the two implementations solve the same declared finite problem to closely matching values.

| Fixture class | Epochs | 3D position RMSE                | Maximum error                   |
|---------------|--------|---------------------------------|---------------------------------|
| Noiseless     | 128    | $3.61 \times 10^{-9} \text{ m}$ | $7.84 \times 10^{-9} \text{ m}$ |
| Noisy         | 122    | 1.6017 m                        | 3.3883 m                        |
| Gross outlier | 1      | 147.0987 m                      | 147.0987 m                      |
| Exactly four  | 1      | $1.23 \times 10^{-8} \text{ m}$ | $1.23 \times 10^{-8} \text{ m}$ |
| 32 channels   | 1      | $5.71 \times 10^{-9} \text{ m}$ | $5.71 \times 10^{-9} \text{ m}$ |

Errors above are measured only against the synthetic generating truth. The nanometre- scale noiseless figures reflect arithmetic roundoff in matched artificial data, not an achievable GNSS hardware accuracy. The noisy fixture demonstrates the selected noise model; a real receiver's error includes its actual correction/model limitations.

Formal covariance, predicted geometry quality and empirical truth error are different quantities. The package keeps them separate, following the distinction made in the positioning-error reference. [G5]

## 28 Prior release verification record

This section retains the 3.6.1.5 preparation record. Current 3.6.1.6 execution evidence is given in the SRK-R1 validation section. The following unexecuted CUDA statements refer to the prior preparation environment only.

| Category                                | Recorded result                        |
|-----------------------------------------|----------------------------------------|
| CPU C++ build and CTest                 | Passed                                 |
| C++ explicit checks                     | 105,559 passed                         |
| Python UGTS/geodesy tests               | 18 passed                              |
| Command-line scenarios                  | 11 passed                              |
| Independent solver comparisons          | 4,828 passed                           |
| CPU Address/UndefinedBehavior Sanitizer | CTest passed                           |
| CUDA compilation                        | not_run: nvcc unavailable              |
| CUDA execution / memory sanitizer       | not_run: no accessible NVIDIA GPU      |
| Windows native build                    | not_run in preparation                 |
| Live GNSS observation validation        | Not performed; synthetic fixtures only |

The C++ count includes all 65,536 possible 16-bit population-count cases and 10,000 independent random word-mask comparisons. It also checks the full JK truth table, QR identities, covariance, rank rejection and the retained whole-word rule. A large assertion count is not a count of separate receivers tested.

The Python suite checks both key codecs against the source's golden pair and 5,000 random tuples, all prefix lengths for the reference key, quantization bounds, full winding, two separate topologies, cone/sweep relations, the jitter-interval distinction, geodetic/ENU round trips and literal OTAN2 semantics.

The CLI scenarios include malformed IDs/channels, negative sigma, NaN input, invalid back-end/options, existing-output protection, a normal run and an iteration-limit run. These prior outcomes are recorded under `results_3_6_1_5/`, with the earlier source test claims kept separate from these actual executions.

In the prior preparation, the device validation script was invoked and recorded its unavailable-tool result. On the laptop it becomes the required next validation step, not a placeholder success. No general CUDA speedup or texture-residency claim is made in this edition.

## 29 Build, run and verify on the requested laptop

Use a CUDA-supported host compiler, CMake 3.24 or newer, and CUDA Toolkit 12.8 or newer for the requested SM120 target. The installed NVIDIA driver must support the chosen runtime/-toolchain. This specifies a minimum architecture-capable toolkit, not a claim that 12.8 is the newest available release. [H2, H3]

```
cmake -S . -B build_gpu -DSATNAV_ENABLE_CUDA=ON \  
  -DCMAKE_BUILD_TYPE=Release  
cmake --build build_gpu --config Release --parallel 2  
ctest --test-dir build_gpu -C Release --output-on-failure  
./build_gpu/satnav --probe  
./build_gpu/satnav --input examples/demo --out run_gpu \  
  --backend cuda --read texture --verify  
python tools/verify_run.py --input examples/demo --run run_gpu  
python tools/ugts_handoff.py --input examples/demo --run run_gpu
```

On a Windows multi-configuration build, use the following executable path:

```
build_gpu/Release/satnav.exe
```

Run configuration from the matching x64 compiler developer terminal so that CMake can locate the host toolchain.

### 29.1 Complete device comparison

```
python tools/validate_gpu.py --sanitizer
```

The script records compilation, device probing, texture/global runs, shared CPU comparisons, independent Python comparisons and requested memcheck results. It uses fresh run directories. Missing required tooling produces a separate unavailable status and exit code 3. A real build/test error remains a failed status.

### 29.2 CPU-only review

```
cmake -S . -B build_cpu -DCMAKE_BUILD_TYPE=Release  
cmake --build build_cpu --config Release --parallel 2  
./build_cpu/satnav --input examples/demo --out run_cpu  
python tools/verify_run.py --input examples/demo --run run_cpu  
python -m unittest discover -s tests -p "test_*.py" -v
```

The Python reference and adapters use the standard library. No PyTorch or external model service is required for this kernel.

### 30 Integration decisions and source issue register

| ID  | Decision                        | Effect                                                                                                                       |
|-----|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| N01 | Corrected-code satnav model     | The GNSS observation equation and solver are explicit new additions, not attributed to U.                                    |
| N02 | Full receiver state retained    | Solve in 3D ECEF plus clock; use a separate local log chart and retain Up and provenance.                                    |
| N03 | Independent QR methods          | Shared CPU/CUDA Givens implementation and independent Python Householder solve; named rank and convergence policy.           |
| N04 | Fixed channel-to-bit mapping    | One epoch is one at-most-32-channel word. Boundary absorption stays whole-word.                                              |
| N05 | Source key layouts preserved    | Exact contiguous/Morton pair retained; quantization ties named; key does not replace full state.                             |
| N06 | Two jitter intervals separated  | Explicit source equality issue documented; perturbation-family and measurement-centred intervals are not silently equated.   |
| N07 | Bounded sweep interpretation    | Translation-only analytic interval; floating arithmetic and nonnested grids not promoted to stronger certificates.           |
| N08 | Topology does not rewrite Earth | Bundle/reflection helpers remain separate local contracts, not GNSS propagation rules.                                       |
| N09 | Computational event semantics   | Declared support/residual policies produce observation records, not a protection level, spoof-proof fix or actuator command. |
| N10 | Recorded hardware evidence      | CUDA was unexecuted in 3.6.1.5 preparation; current device builds, replay and memcheck results are recorded separately.      |
| N11 | Explicit literal feedback       | SRK-R1 composes ASA/NA and JK with supplied Boolean expressions, synchronous state carry and present-mask projection.        |

These decisions define the explicit profiles of aTOMos 3.6.1 with the retained UGTS prototype integrated at explicit interfaces. They do not discard the user's operator vocabulary or pretend that all missing GNSS physics were already encoded in a name.

A later variant can extend the estimator with new states, observable types or correction adapters. It must identify the changed observation model and preserve a projection or compatibility test to this profile where appropriate. An unreported inter-system bias, new residual exclusion rule or changed topology is not a transparent optimization.

## 31 Source and artifact inventory

**U.** *UGTS\_KC\_3\_6\_2\_SCLP\_Tom\_Klootwijk(1).pdf*. The file has 19 physical pages and a 16-page numbered main/appendix sequence. The source's referenced 13-page precursor and software ZIP were not supplied. The inherited source manifest records its PDF SHA-256 as

```
6987490a0a7b7ed76b0091057a2e69ab831fee786434a0b4219782923f5ea580
```

The U page references in this formalization point to the printed main numbers. For example, its key table on printed p. 8 is physical PDF page 11. The relevant figure, table and jitter-equation pages were inspected as rendered pages as well as text.

**A.** aTOMos v3.6.1 / M2 source and code contract in this project conversation. The prior archive was not mounted in this turn; the specialization preserves the written ASA/JK/OTAN2 contracts rather than claiming a binary patch to an unavailable artifact. The actual new source is included and tested to the extent recorded above.

| Delivered file family                        | Purpose                                                                                                                                                                                           |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| include/, src/, cuda/<br>python/             | Shared solver, executable and real CUDA backend source.<br>Independent solution and source-derived UGTS/geodesy helpers.                                                                          |
| examples/demo/<br>tests/, results/<br>tools/ | Prepared artificial epochs, observations, truth and profile.<br>Reproducible checks and the actual recorded outcomes.<br>Build/review, independent comparison, event adapter and integrity tools. |
| docs/                                        | This editable formalization, input contract and source-decision register.                                                                                                                         |
| source/manifest.json                         | Exact source availability and fingerprint record.                                                                                                                                                 |
| SHA256SUMS.txt                               | Delivered-byte integrity manifest.                                                                                                                                                                |

Source font files and proprietary binaries are not distributed. The current uploaded UGTS reference may be retained separately for source comparison; its equations and selected mechanisms are integrated into this volume instead of concatenating unrelated older transcripts into the kernel package.

## Primary implementation references

Recorded by the 3.6.1.5 preparation, 14 September 2026. These external references support the satnav model and hardware contract; they are separate from the user's source corpus.

- G1 ESA Navipedia, J. Sanz Subirana, J.M. Juan Zornoza and M. Hernández-Pajares, [Code Based Positioning \(SPS\)](#). Four-state model, correction terms, linearized geometry and iterative position update.
- G2 ESA Navipedia, [Satellite Coordinates Computation](#). Emission-time satellite state and reception-time frame treatment.
- G3 ESA Navipedia, [Transformations between ECEF and ENU coordinates](#). Local frame conversion and angular conventions.
- G4 ESA Navipedia, [Parameters adjustment](#). Linearized residual and least-squares framework.
- G5 ESA Navipedia, [Positioning Error](#). Formal covariance, predicted geometry and measured error distinguished.
- G6 NGA Office of Geomatics, [World Geodetic System 1984](#). Ellipsoid/reference-system definition and constants used in the coordinate utility.
- H1 NVIDIA, [GeForce RTX 50 Series Laptops](#). RTX 5070 Ti Laptop memory specification.
- H2 NVIDIA, [CUDA GPU Compute Capability](#). GeForce RTX 5070 Ti family listed at compute capability 12.0.
- H3 NVIDIA, [CUDA 12.8 Features Archive](#). Introduction of SM\_120 compiler support.
- H4 NVIDIA, [CUDA Runtime API: Texture Object Management](#). Linear resource and element-read contract.

### aTOMos v3.6.1.6 — Literal self-referential kernel

Tom Klootwijk NL200678942 10-07-1990

**Profiles:** SRK-R1 and SATNAV-R1. **Release date:** 14 September 2026.

Literal self-reference with editable equations, CPU/CUDA source, independent replay, and the retained SATNAV/UGTS formalization. Prepared with AI assistance.